

Laboratorio 3

Integrantes: Andrea Arias, Matthew Flores, Johan Wang

Repositorio de Git con los scripts: <https://github.com/andrearj19/Laboratorio-3>

1. Introducción

Para realizar este laboratorio se reutiliza la base de datos creada para Industria Lojana de Especerías (ILE) en los laboratorios del primer bimestre.

La base de datos implementa clientes, proveedores, materias primas, compras, productos, trabajadores, producción y ventas. Asimismo, contiene las tablas de detalle que permiten el registro de las materias primas compradas; los insumos utilizados en producción y los productos que son objeto de la venta.

Algunas de las principales transacciones reconocidas con anterioridad son:

Consultar el inventario de materias primas. Consultar las compras por proveedor. Consultar los productos terminados disponibles. Consultar la producción por periodo. Consultar las ventas por cliente. Consultar los productos más vendidos. Registrar las compras y su detalle. Registrar los procesos de producción. Registrar las ventas. Actualizar las existencias de materias primas y de productos.

Sobre esas transacciones se aplicarán funciones de SQL avanzado, programación PL/SQL y técnicas de optimización.

2. Aplicación de consultas SQL avanzadas

2.1. Subconsulta: productos vendidos por encima del promedio

Esta consulta identifica los productos cuya cantidad total vendida es superior al promedio de ventas de todos los productos.

The screenshot shows a SQL query in a 'Generador de Consultas' (Query Generator) window. The query is as follows:

```

SELECT
    p.id_producto,
    p.nombre,
    SUM(dv.cantidad) AS total_unidades_vendidas
FROM producto p
JOIN detalle_venta dv
    ON p.id_producto = dv.id_producto
GROUP BY
    p.id_producto,
    p.nombre
HAVING SUM(dv.cantidad) > (
    SELECT AVG(total_producto)
    FROM (
        SELECT SUM(cantidad) AS total_producto
        FROM detalle_venta
        GROUP BY id_producto
    )
)
ORDER BY total_unidades_vendidas DESC;

```

Below the query, the 'Resultado de la Consulta' (Query Result) window shows the results of the query. It indicates that 2 rows were recovered in 0.019 seconds. The results are displayed in a table with three columns: ID_PRODUCTO, NOMBRE, and TOTAL_UNIDADES_VENDIDAS.

ID_PRODUCTO	NOMBRE	TOTAL_UNIDADES_VENDIDAS
1	1 Orégano molido ILE 50g	35
2	2 Comino molido ILE 50g	14

Explicación: La subconsulta interna calcula la cantidad total vendida de cada producto. Posteriormente, la segunda subconsulta obtiene el promedio de dichas cantidades.

La consulta principal compara el total vendido de cada producto con ese promedio.

Beneficio obtenido: Permite identificar los productos con una demanda superior al comportamiento normal de ventas. Esto ayuda a tomar decisiones relacionadas con:

- Incrementar la producción.
- Evitar falta de existencias.
- Planificar promociones.
- Priorizar la compra de materias primas.

2.2. Agrupamiento: total de ventas por cliente

```
--Agrupamiento total de ventas por cliente--
SELECT
    c.id_cliente,
    c.nombres || ' ' || c.apellidos AS cliente,
    COUNT(v.id_venta) AS cantidad_ventas,
    SUM(v.total) AS total_comprado,
    ROUND(AVG(v.total), 2) AS promedio_por_venta
FROM cliente c
JOIN venta v
    ON c.id_cliente = v.id_cliente
GROUP BY
    c.id_cliente,
    c.nombres,
    c.apellidos
ORDER BY total_comprado DESC;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 4 en 0,053 segundos

ID_CLIENTE	CLIENTE	CANTIDAD_VENTAS	TOTAL_COMPRADO	PROMEDIO_POR_VENTA
1	1 Ana Gómez	2	37,7	18,85
2	3 Maria Pérez	1	20,25	20,25
3	4 Carlos Torres	1	16,5	16,5
4	2 Luis Maldonado	1	15,4	15,4

Explicación: La consulta agrupa las ventas según el cliente y utiliza funciones de agregación:

COUNT: cuenta las ventas realizadas.

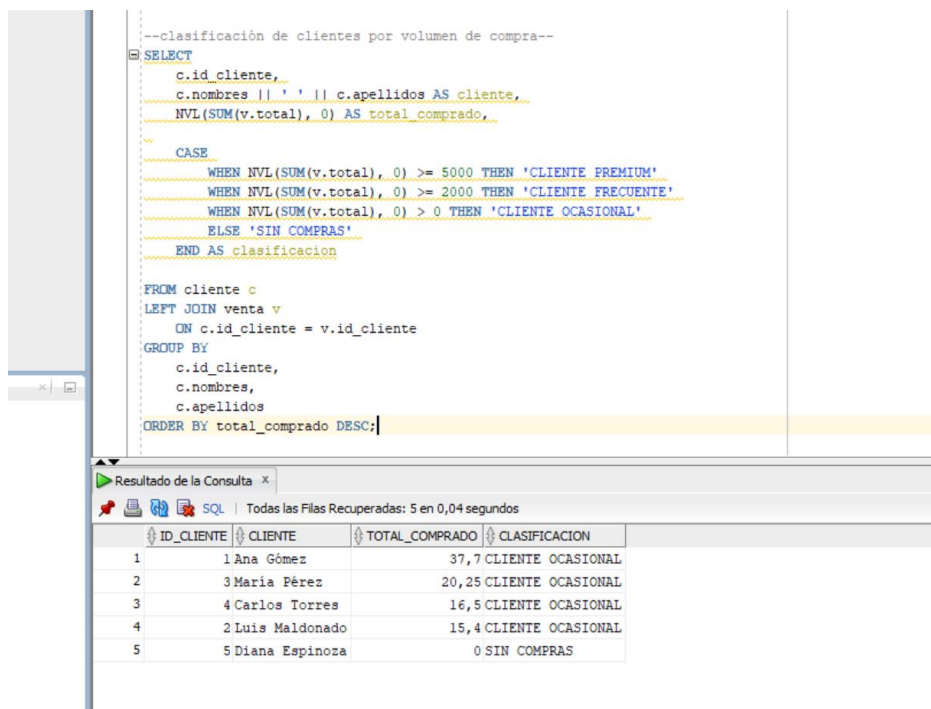
SUM: calcula el monto total comprado.

AVG: calcula el valor promedio de cada venta.

ROUND: redondea el resultado a dos decimales.

Beneficio obtenido: Facilita la identificación de los clientes que generan mayores ingresos para la empresa y permite desarrollar estrategias de fidelización.

2.3. CASE: clasificación de clientes por volumen de compra



```
--clasificación de clientes por volumen de compra--
SELECT
  c.id_cliente,
  c.nombres || ' ' || c.apellidos AS cliente,
  NVL(SUM(v.total), 0) AS total_comprado,
  CASE
    WHEN NVL(SUM(v.total), 0) >= 5000 THEN 'CLIENTE PREMIUM'
    WHEN NVL(SUM(v.total), 0) >= 2000 THEN 'CLIENTE FRECUENTE'
    WHEN NVL(SUM(v.total), 0) > 0 THEN 'CLIENTE OCASIONAL'
    ELSE 'SIN COMPRAS'
  END AS clasificacion
FROM cliente c
LEFT JOIN venta v
  ON c.id_cliente = v.id_cliente
GROUP BY
  c.id_cliente,
  c.nombres,
  c.apellidos
ORDER BY total_comprado DESC;
```

ID_CLIENTE	CLIENTE	TOTAL_COMPRA	CLASIFICACION
1	1 Ana Gómez	37,7	CLIENTE OCASIONAL
2	3 Maria Pérez	20,25	CLIENTE OCASIONAL
3	4 Carlos Torres	16,5	CLIENTE OCASIONAL
4	2 Luis Maldonado	15,4	CLIENTE OCASIONAL
5	5 Diana Espinoza	0	SIN COMPRAS

Explicación: La expresión CASE clasifica a los clientes según el monto total de sus compras:

Total comprado y Clasificación

Total comprado	Clasificación
\$5.000 o más	Cliente premium
Entre \$2.000 y \$4.999,99	Cliente frecuente
Menos de \$2.000	Cliente ocasional
Sin ventas	Sin compras

La función NVL reemplaza los valores nulos por cero.

Beneficio obtenido: Transforma valores numéricos en categorías comprensibles para los usuarios administrativos. Esto facilita la segmentación comercial sin modificar los datos originales.

2.4. DECODE: estado del inventario de productos

```
-- estado del inventario de productos--
SELECT
    id_producto,
    nombre,
    stock,

    DECODE(
        SIGN(stock),
        -1, 'STOCK INCORRECTO',
        0, 'AGOTADO',
        'DISPONIBLE'
    ) AS estado_general
FROM producto
ORDER BY stock ASC;
```

ID_PRODUCTO	NOMBRE	STOCK	ESTADO_GENERAL
1	6 Salsa de ají ILE	150	DISPONIBLE
2	5 Canela molida 50g	200	DISPONIBLE
3	4 Ajo en polvo ILE 80g	250	DISPONIBLE
4	3 Pimienta negra molida 60g	300	DISPONIBLE
5	2 Comino molido ILE 50g	350	DISPONIBLE
6	1 Orégano molido ILE 50g	500	DISPONIBLE

Explicación

La función SIGN devuelve:

- -1 si el número es negativo.
- 0 si el número es cero.
- 1 si el número es positivo.

Posteriormente, DECODE transforma esos resultados en una descripción.

Beneficio obtenido: Permite interpretar rápidamente el estado general del inventario, especialmente para detectar productos agotados o registros incorrectos.

2.5. CASE para una clasificación más detallada del stock

--CASE para una clasificación más detallada del stock--			
<pre> SELECT id_producto, nombre, stock, CASE WHEN stock = 0 THEN 'AGOTADO' WHEN stock BETWEEN 1 AND 20 THEN 'STOCK CRÍTICO' WHEN stock BETWEEN 21 AND 50 THEN 'STOCK BAJO' ELSE 'STOCK SUFICIENTE' END AS estado_stock FROM producto ORDER BY stock; </pre>			
Resultado de la Consulta x			
 Todas las Filas Recuperadas: 6 en 0,004 segundos			
ID_PRODUCTO	NOMBRE	STOCK	ESTADO_STOCK
1	6 Salsa de ají ILE	150	STOCK SUFICIENTE
2	5 Canela molida 50g	200	STOCK SUFICIENTE
3	4 Ajo en polvo ILE 80g	250	STOCK SUFICIENTE
4	3 Pimienta negra molida 60g	300	STOCK SUFICIENTE
5	2 Comino molido ILE 50g	350	STOCK SUFICIENTE
6	1 Orégano molido ILE 50g	500	STOCK SUFICIENTE

Beneficio obtenido: Permite detectar anticipadamente los productos que necesitan ser fabricados nuevamente antes de que se agoten.

2.6. Subconsulta correlacionada: última venta de cada cliente

```
--Subconsulta correlacionada: última venta de cada cliente--
```

```
SELECT
  c.id_cliente,
  c.nombres || ' ' || c.apellidos AS cliente,
  (
    SELECT MAX(v.fecha)
    FROM venta v
    WHERE v.id_cliente = c.id_cliente
  ) AS ultima_fecha_compra
FROM cliente c
ORDER BY ultima_fecha_compra DESC NULLS LAST;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 5 en 0,052 segundos

ID_CLIENTE	CLIENTE	ULTIMA_FECHA_COMPRA
1	4 Carlos Torres	30/05/2026
2	1 Ana Gómez	22/04/2026
3	3 María Pérez	15/03/2026
4	2 Luis Maldonado	28/02/2026
5	5 Diana Espinoza	(null)

Explicación: La subconsulta se ejecuta relacionando cada cliente con sus propias ventas.

Beneficio obtenido: Permite identificar clientes inactivos o clientes que no han comprado recientemente, lo que puede utilizarse para campañas de recuperación.

2.7. Agrupamiento por mes: ingresos mensuales

```
--Agrupamiento por mes: ingresos mensuales--
```

```
SELECT
  TO_CHAR(fecha, 'YYYY-MM') AS periodo,
  COUNT(id_venta) AS numero_ventas,
  SUM(total) AS ingreso_total,
  ROUND(AVG(total), 2) AS promedio_venta
FROM venta
GROUP BY TO_CHAR(fecha, 'YYYY-MM')
ORDER BY periodo;
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 5 en 0,042 segundos

PERIODO	NUMERO_VENTAS	INGRESO_TOTAL	PROMEDIO_VENTA
1 2026-01	1	18,5	18,5
2 2026-02	1	15,4	15,4
3 2026-03	1	20,25	20,25
4 2026-04	1	19,2	19,2
5 2026-05	1	16,5	16,5

Beneficio obtenido: Permite analizar la evolución mensual de los ingresos y reconocer períodos de mayor o menor actividad comercial.

2.8. Productos más vendidos utilizando una subconsulta

```
--Productos mas vendidos utilizando subconsulta--
SELECT
    p.id_producto,
    p.nombre,
    SUM(dv.cantidad) AS unidades_vendidas
FROM producto p
JOIN detalle_venta dv
    ON p.id_producto = dv.id_producto
GROUP BY
    p.id_producto,
    p.nombre
HAVING SUM(dv.cantidad) = (
    SELECT MAX(total_vendido)
    FROM (
        SELECT SUM(cantidad) AS total_vendido
        FROM detalle_venta
        GROUP BY id_producto
    )
);
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 1 en 0,031 segundos

ID_PRODUCTO	NOMBRE	UNIDADES_VENDIDAS
1	1 Orégano molido ILE 50g	35

Beneficio obtenido: Devuelve directamente el producto o los productos que ocupan el primer lugar en ventas, incluso cuando existe un empate.

2.9. Clasificación de proveedores con CASE

```
-- Clasificación de proveedores con CASE--
SELECT
    pr.id_proveedor,
    pr.nombre_company AS nombre_proveedor,
    COUNT(c.id_compra) AS numero_compras,
    NVL(SUM(c.total), 0) AS monto_comprado,
    CASE
        WHEN NVL(SUM(c.total), 0) >= 10000
        THEN 'PROVEEDOR ESTRATÉGICO'
        WHEN NVL(SUM(c.total), 0) >= 5000
        THEN 'PROVEEDOR FRECUENTE'
        WHEN NVL(SUM(c.total), 0) > 0
        THEN 'PROVEEDOR OCASIONAL'
        ELSE 'SIN COMPRAS'
    END AS tipo_proveedor
FROM proveedor pr
```

Resultado de la Consulta x

Todas las Filas Recuperadas: 3 en 0,003 segundos

ID_PROVEEDOR	NOMBRE_PROVEEDOR	NUMERO_COMPRAS	MONTO_COMPRADO	TIPO_PROVEEDOR
1	1 Agrícola del Austro S.A.	1	1825	PROVEEDOR OCASIONAL
2	2 Especies Andinas Cia. Ltda.	1	1550	PROVEEDOR OCASIONAL
3	3 Productos Naturales Loja	1	1200	PROVEEDOR OCASIONAL

Beneficio obtenido: Facilita la selección y evaluación de proveedores según el volumen económico de las compras realizadas.

2.10. Consulta de producción por período y empleado

<pre> --Consulta de produccion por periodo y empleado-- SELECT e.id_empleado, e.nombre AS empleado, e.cargo, COUNT(pr.id_produccion) AS lotes_realizados, SUM(pr.cantidad_producida) AS unidades_producidas, ROUND(AVG(pr.cantidad_producida), 2) AS promedio_por_lote FROM empleado e JOIN produccion pr ON e.id_empleado = pr.id_empleado WHERE pr.fecha BETWEEN TO_DATE('01/01/2026', 'DD/MM/YYYY') AND TO_DATE('31/12/2026', 'DD/MM/YYYY') GROUP BY e.id_empleado, e.nombre, e.cargo ORDER BY unidades_producidas DESC; </pre>																							
Resultado de la Consulta X Todas las Filas Recuperadas: 2 en 0,025 segundos <table> <thead> <tr> <th>ID_EMPLEADO</th><th>EMPLEADO</th><th>CARGO</th><th>LOTES_REALIZADOS</th><th>UNIDADES_PRODUCIDAS</th><th>PROMEDIO_POR_LOTE</th></tr> </thead> <tbody> <tr> <td>1</td><td>Carlos Armijos</td><td>Operador de Producción A</td><td>3</td><td>1100</td><td>366,67</td></tr> <tr> <td>2</td><td>Diana Espinoza</td><td>Operador de Producción B</td><td>2</td><td>700</td><td>350</td></tr> </tbody> </table>						ID_EMPLEADO	EMPLEADO	CARGO	LOTES_REALIZADOS	UNIDADES_PRODUCIDAS	PROMEDIO_POR_LOTE	1	Carlos Armijos	Operador de Producción A	3	1100	366,67	2	Diana Espinoza	Operador de Producción B	2	700	350
ID_EMPLEADO	EMPLEADO	CARGO	LOTES_REALIZADOS	UNIDADES_PRODUCIDAS	PROMEDIO_POR_LOTE																		
1	Carlos Armijos	Operador de Producción A	3	1100	366,67																		
2	Diana Espinoza	Operador de Producción B	2	700	350																		

Beneficio obtenido: Permite evaluar la producción realizada por cada empleado dentro de un período determinado.

3. Beneficio general del SQL avanzado

La aplicación de subconsultas, agrupamientos, CASE, DECODE y funciones de agregación aporta los siguientes beneficios:

- Reduce el procesamiento manual, porque los cálculos se realizan directamente en Oracle.
- Presenta información resumida, evitando revisar cientos o miles de registros individuales.
- Facilita la toma de decisiones, mediante clasificaciones como stock crítico, cliente premium o proveedor estratégico.

4. Procedimiento almacenado

4.1. Procedimiento para registrar el detalle de una venta

Se selecciona la transacción de actualización denominada registrar una venta y actualizar el stock de productos terminados. En el laboratorio anterior esta transacción afectaba las tablas VENTA, DETALLE_VENTA y PRODUCTO.

0,029 segundos

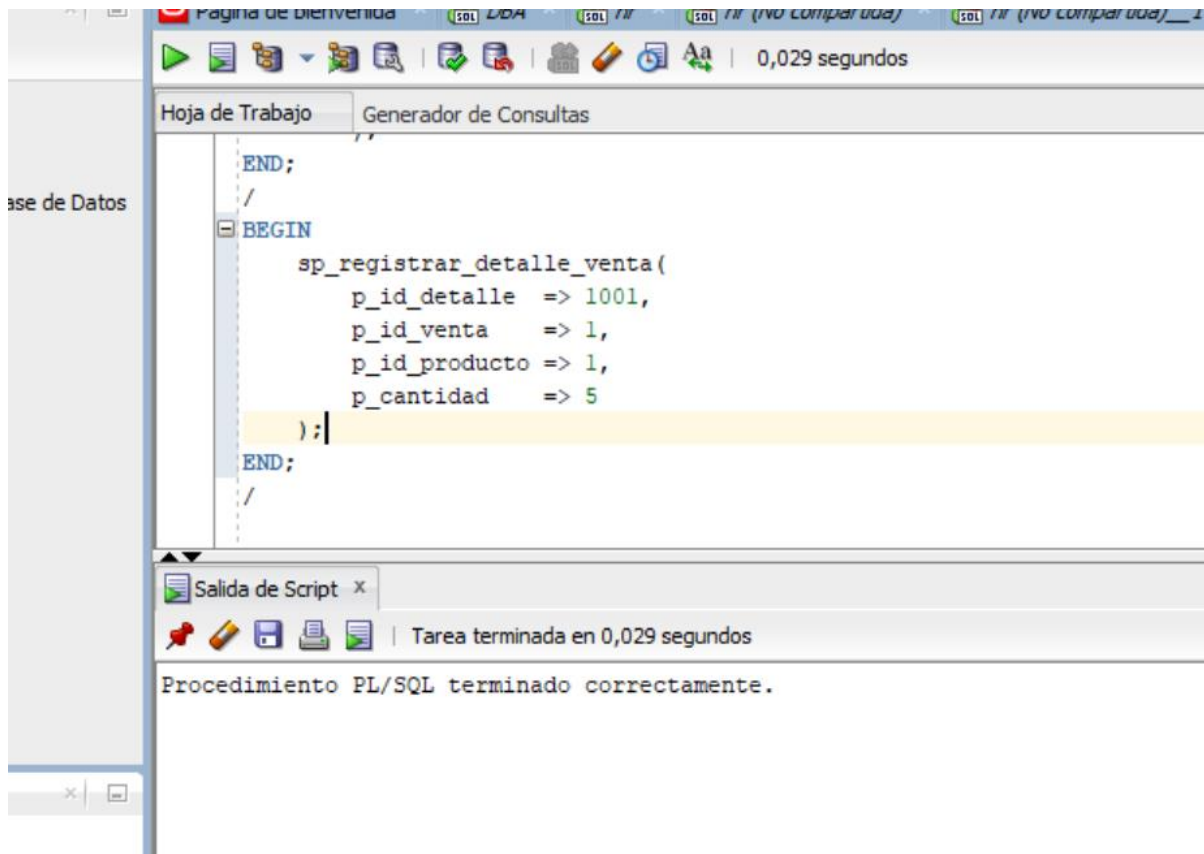
Hoja de Trabajo | Generador de Consultas

--Procedimiento para registrar el detalle de venta--
CREATE OR REPLACE PROCEDURE sp_registrar_detalle_venta (
 p_id_detalle IN detalle_venta.id_detalle%TYPE,
 p_id_venta IN detalle_venta.id_venta%TYPE,
 p_id_producto IN detalle_venta.id_producto%TYPE,
 p_cantidad IN detalle_venta.cantidad%TYPE
)
IS
 v_precio producto.precio%TYPE;
 v_stock producto.stock%TYPE;
BEGIN
 IF p_cantidad <= 0 THEN
 RAISE_APPLICATION_ERROR(
 -20001,
 'La cantidad vendida debe ser mayor que cero.'
);
 END IF;

 SELECT precio, stock
 INTO v_precio, v_stock
 FROM producto
 WHERE id_producto = p_id_producto
 FOR UPDATE;

 IF v_stock < p_cantidad THEN
 RAISE_APPLICATION_ERROR(
 -20002,
 'Stock insuficiente. Stock disponible: ' || v_stock
);
 END IF;

 INSERT INTO detalle_venta (
 id_detalle,
 cantidad,
 precio_unitario,
 subtotal,
 id_venta,
 id_producto
)
VALUES (
 p_id_detalle,
 p cantidad.



Beneficio en cuanto a ejecución

El procedimiento:

- Ejecuta el registro y la actualización del inventario dentro del servidor.
- Evita enviar varias instrucciones separadas desde la aplicación.
- Utiliza FOR UPDATE para bloquear temporalmente el producto mientras se valida y modifica el stock.
- Reduce el riesgo de ventas duplicadas o cantidades negativas.

5. Función almacenada

5.1. Función para calcular el total de una venta

```
Hoja de Trabajo | Generador de Consultas

--Funcion para calcular el total de una venta--
CREATE OR REPLACE FUNCTION fn_total_venta (
  p_id_venta IN venta.id_venta%TYPE
)
RETURN NUMBER
IS
  v_total NUMBER(12,2);
BEGIN
  SELECT NVL(SUM(subtotal), 0)
  INTO v_total
  FROM detalle_venta
  WHERE id_venta = p_id_venta;

  RETURN v_total;
END;
/
SELECT
  id_venta,
  fn_total_venta(id_venta) AS total_calculado
FROM venta;
```

Salida de Script x | Resultado de la Consulta x

Tarea terminada en 0,059 segundos

Function FN_TOTAL_VENTA compilado

```
SELECT
  id_venta,
  fn_total_venta(id_venta) AS total_calculado
FROM venta;
```

Salida de Script x | Resultado de la Consulta x

Todas las Filas Recuperadas: 5 en 0,024 segundos

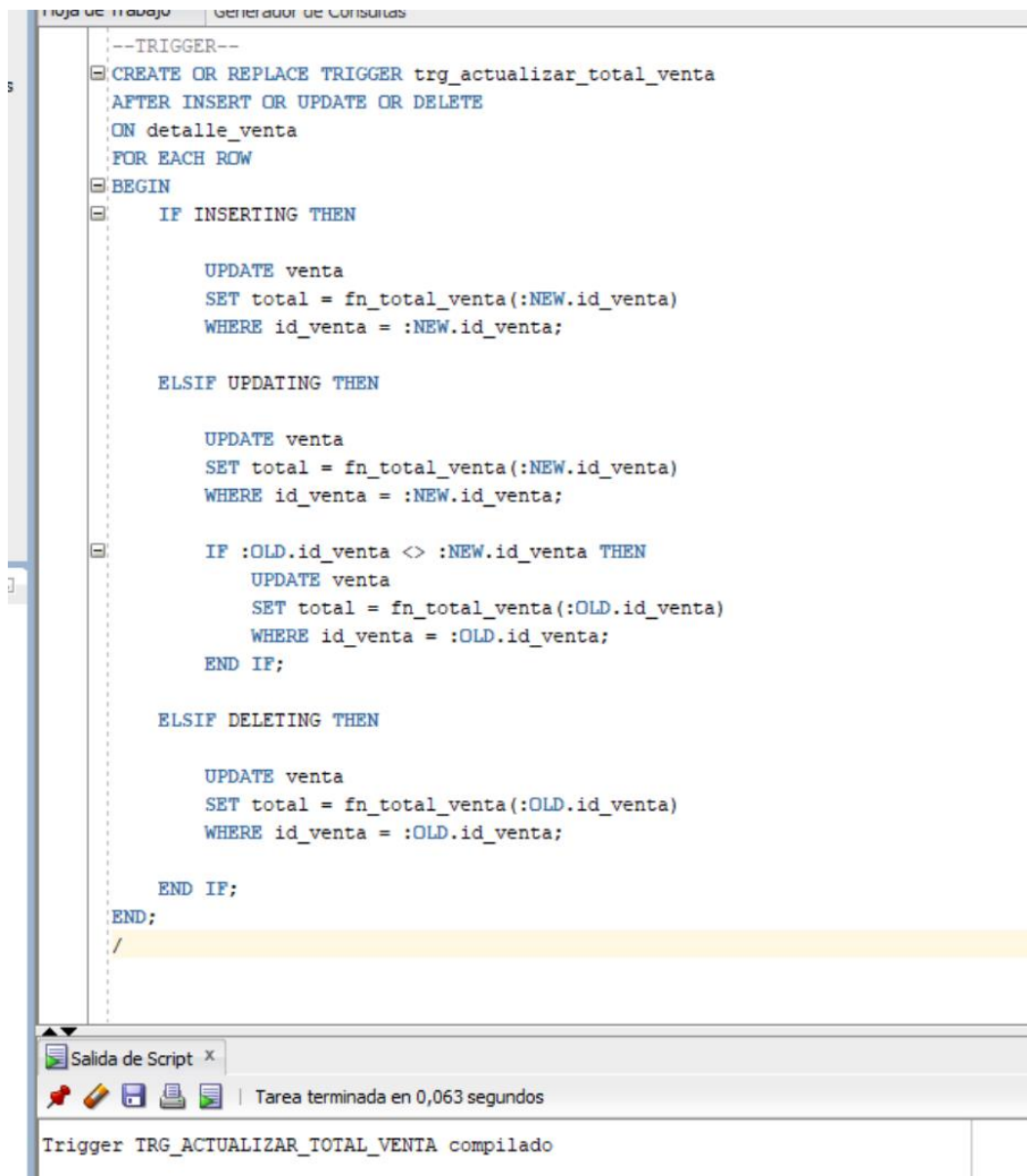
ID_VENTA	TOTAL_CALCULADO
1	18,5
2	15,4
3	20,25
4	19,2
5	16,5

Beneficio en cuanto a ejecución: La función permite reutilizar el cálculo del total de una venta desde: Consultas. Procedimientos. Reportes. Aplicaciones. Triggers.

6. Trigger

6.1. Trigger para mantener actualizado el total de la venta

El siguiente trigger actualiza automáticamente el campo total de la tabla VENTA cuando se inserta, modifica o elimina un detalle.



```
--TRIGGER--
CREATE OR REPLACE TRIGGER trg_actualizar_total_venta
AFTER INSERT OR UPDATE OR DELETE
ON detalle_venta
FOR EACH ROW
BEGIN
    IF INSERTING THEN

        UPDATE venta
        SET total = fn_total_venta(:NEW.id_venta)
        WHERE id_venta = :NEW.id_venta;

    ELSIF UPDATING THEN

        UPDATE venta
        SET total = fn_total_venta(:NEW.id_venta)
        WHERE id_venta = :NEW.id_venta;

        IF :OLD.id_venta <> :NEW.id_venta THEN
            UPDATE venta
            SET total = fn_total_venta(:OLD.id_venta)
            WHERE id_venta = :OLD.id_venta;
        END IF;

    ELSIF DELETING THEN

        UPDATE venta
        SET total = fn_total_venta(:OLD.id_venta)
        WHERE id_venta = :OLD.id_venta;

    END IF;
END;
```

Salida de Script x

Tarea terminada en 0,063 segundos

Trigger TRG_ACTUALIZAR_TOTAL_VENTA compilado

Beneficio en cuanto a ejecución: El trigger se ejecuta automáticamente y evita que el usuario tenga que actualizar manualmente el total de la venta. Esto proporciona:

Mayor consistencia, Menor posibilidad de errores, Actualización inmediata.

8. Prueba completa del procedimiento, función y trigger

```
--Paso 1. Registrar la cabecera de la venta--
INSERT INTO venta (
    id_venta,
    fecha,
    total,
    id_cliente
)
VALUES (
    502,
    SYSDATE,
    0,
    1
);

COMMIT;
```

Salida de Script x

Tarea terminada en 0,041 segundos

1 fila insertadas.

Confirmación terminada.

```
--Ejecutar el procedimiento--
BEGIN
    sp_registrar_detalle_venta(
        p_id_detalle => 1001,
        p_id_venta   => 502,
        p_id_producto => 1,
        p_cantidad   => 5
    );
END;
/

COMMIT;
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0,043 segundos

Trigger TRG_ACTUALIZAR_TOTAL_VENTA compilado

Procedimiento PL/SQL terminado correctamente.

Confirmación terminada.

```
--Comprobar el detalle--
SELECT *
FROM detalle_venta
WHERE id_venta = 502;
```

Salida de Script x Resultado de la Consulta x

   SQL | Todas las Filas Recuperadas: 1 en 0,006 segundos

ID_DETALLE	CANTIDAD	PRECIO_UNITARIO	SUBTOTAL	ID_VENTA	ID_PRODUCTO	
1	1001	5	1,25	6,25	502	1

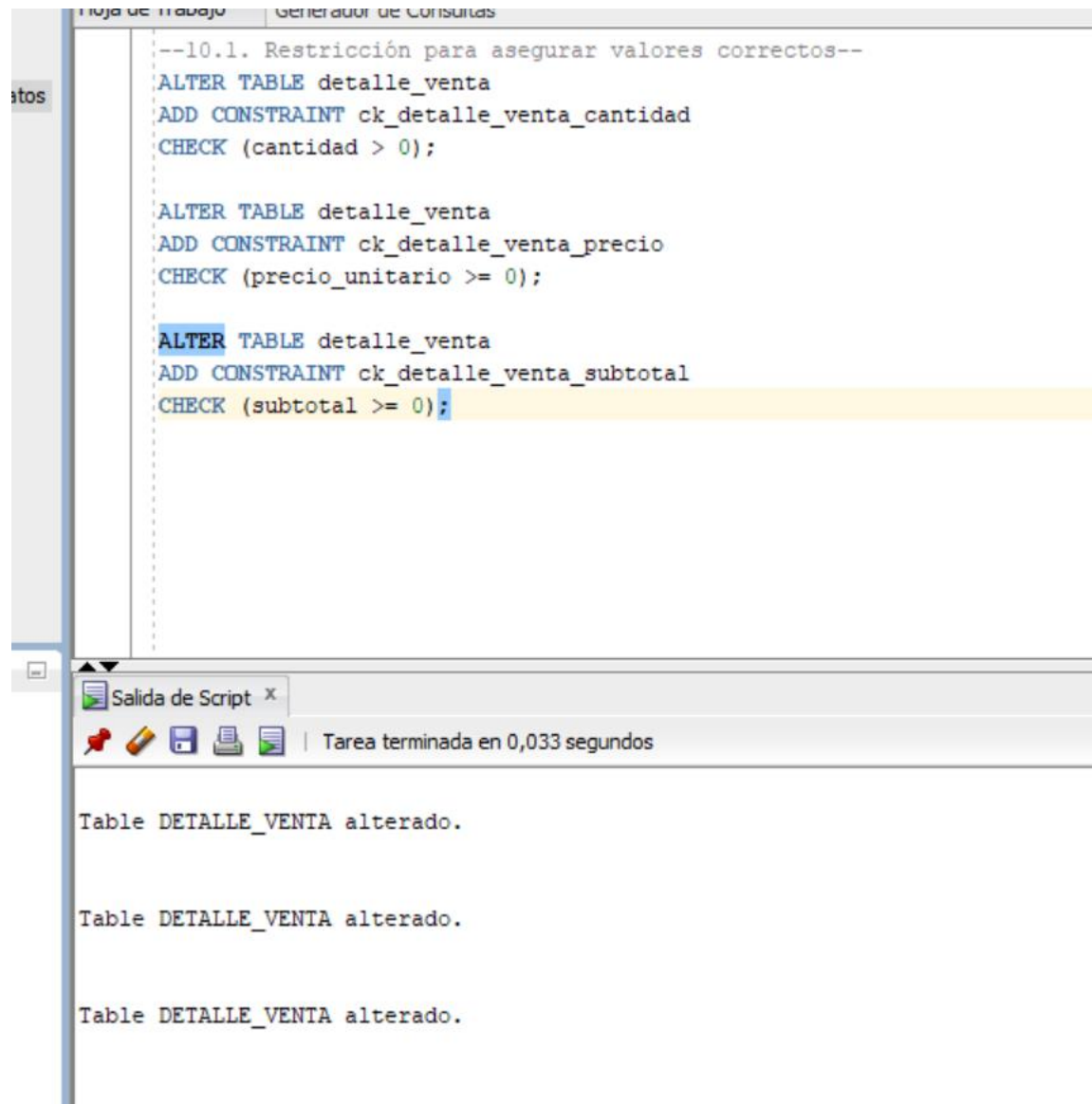
```
--Comprobar el total actualizado--
SELECT id_venta, fecha, total, id_cliente
FROM venta
WHERE id_venta = 502;
```

Salida de Script x		Resultado de la Consulta x	
		SQL Todas las Filas Recuperadas: 1 en 0,004 segundos	
ID_VENTA	FECHA	TOTAL	ID_CLIENTE
1	502 22/07/2026	6,25	1

```
--Comprobar el stock--
SELECT id_producto, nombre, precio, stock
FROM producto
WHERE id_producto = 1;
```

Salida de Script x		Resultado de la Consulta x	
		SQL Todas las Filas Recuperadas: 1 en 0,002 segundos	
ID_PRODUCTO	NOMBRE	PRECIO	STOCK
1	1 Orégano molido ILE 50g	1,25	495

9. Aplicación de redundancia controlada en ventas



```
--10.1. Restricción para asegurar valores correctos--
ALTER TABLE detalle_venta
ADD CONSTRAINT ck_detalle_venta_cantidad
CHECK (cantidad > 0);

ALTER TABLE detalle_venta
ADD CONSTRAINT ck_detalle_venta_precio
CHECK (precio_unitario >= 0);

ALTER TABLE detalle_venta
ADD CONSTRAINT ck_detalle_venta_subtotal
CHECK (subtotal >= 0);
```

Salida de Script x

Tarea terminada en 0,033 segundos

Table DETALLE_VENTA alterado.

Table DETALLE_VENTA alterado.

Table DETALLE_VENTA alterado.

10.2. Trigger para calcular automáticamente el subtotal


```
CREATE OR REPLACE TRIGGER trg_calcular_subtotal_venta
BEFORE INSERT OR UPDATE OF cantidad, precio_unitario
ON detalle_venta
FOR EACH ROW
BEGIN
    IF :NEW.cantidad <= 0 THEN
        RAISE_APPLICATION_ERROR(
            -20010,
            'La cantidad debe ser mayor que cero.'
        );
    END IF;

    IF :NEW.precio_unitario < 0 THEN
        RAISE_APPLICATION_ERROR(
            -20011,
            'El precio unitario no puede ser negativo.'
        );
    END IF;
END;
```

Salida de Script x

Tarea terminada en 0,045 segundos

Table DETALLE_VENTA alterado.

Table DETALLE_VENTA alterado.

Table DETALLE_VENTA alterado.

Trigger TRG_CALCULAR_SUBTOTAL_VENTA compilado

Beneficio: El usuario no escribe manualmente el subtotal, evitando inconsistencias como:

Cantidad: 5

Precio: 2,00

Subtotal ingresado incorrectamente: 8,00

El trigger siempre almacenará:

Subtotal: 10,00

10. Redundancia controlada en compras

```
FOR EACH ROW
BEGIN
    IF :NEW.cantidad <= 0 THEN
        RAISE_APPLICATION_ERROR(
            -20020,
            'La cantidad comprada debe ser mayor que cero.'
        );
    END IF;

    IF :NEW.precio_unitario < 0 THEN
        RAISE_APPLICATION_ERROR(
            -20021,
            'El precio unitario no puede ser negativo.'
        );
    END IF;

    :NEW.subtotal := :NEW.cantidad * :NEW.precio_unitario;
END;
```

Salida de Script x

Tarea terminada en 0,046 segundos

Trigger TRG_CALCULAR_SUBTOTAL_COMPRA compilado

Función para calcular el total de una compra

```
--Función para calcular el total de una compra--
CREATE OR REPLACE FUNCTION fn_total_compra (
    p_id_compra IN compra.id_compra%TYPE
)
RETURN NUMBER
IS
    v_total NUMBER(12,2);
BEGIN
    SELECT NVL(SUM(subtotal), 0)
    INTO v_total
    FROM detalle_compra
    WHERE id_compra = p_id_compra;

    RETURN v_total;
END;
```

Salida de Script x

Tarea terminada en 0,043 segundos

Trigger TRG_CALCULAR_SUBTOTAL_COMPRA compilado

Function FN_TOTAL_COMPRA compilado

11. Creación de índices como técnica adicional de afinamiento

```

--Índice para ventas por cliente y fecha--
CREATE INDEX idx_venta_cliente_fecha
ON venta(id_cliente, fecha);
--Consulta--
SELECT *
FROM venta
WHERE id_cliente = 15
AND fecha BETWEEN
    TO_DATE('01/01/2026', 'DD/MM/YYYY')
    AND TO_DATE('31/12/2026', 'DD/MM/YYYY');

--Índice para buscar detalles por producto--
CREATE INDEX idx_detalle_venta_producto
ON detalle_venta(id_producto);
--Consulta--
SELECT SUM(cantidad)
FROM detalle_venta
WHERE id_producto = 10;

--Índice para compras por proveedor y fecha--
CREATE INDEX idx_compra_proveedor_fecha
ON compra(id_proveedor, fecha);
--Consulta--
SELECT *
FROM compra
WHERE id_proveedor = 5
ORDER BY fecha;

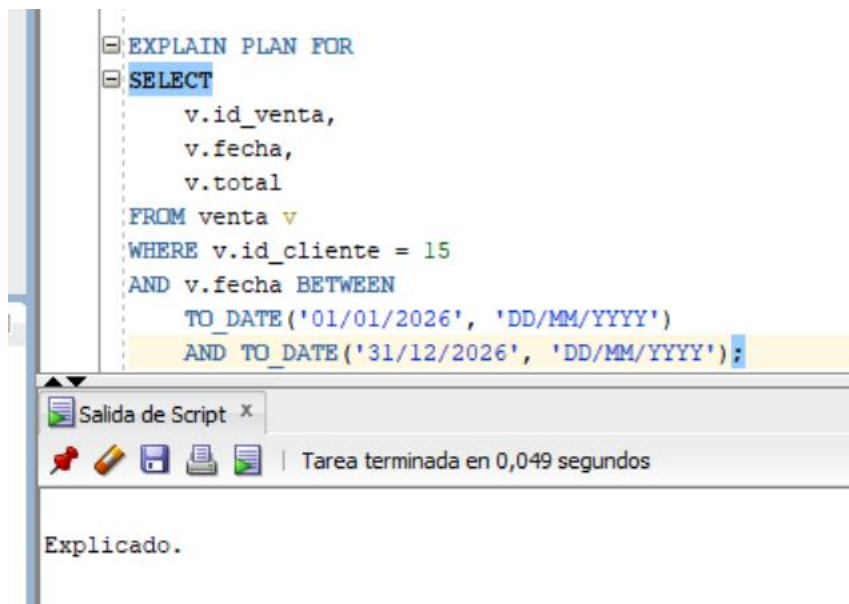
--Índice para producción por producto y fecha--
CREATE INDEX idx_produccion_producto_fecha
ON produccion(id_producto, fecha);

```

Beneficio general de los índices

- Reducen el número de bloques que Oracle debe leer.
- Mejoran búsquedas por cliente, producto, proveedor y fecha.
- Aceleran uniones entre tablas.

12. Verificación del plan de ejecución



SELECT *	
FROM TABLE(DBMS_XPLAN.DISPLAY);	
Salida de Script x Resultado de la Consulta x	
SQL Todas las Filas Recuperadas: 19 en 0,249 segundos	
PLAN_TABLE_OUTPUT	
1 Plan hash value: 862226374	
2	
3	
4 Id Operation Name Rows Bytes Cost (%CPU) Time	
5	
6 0 SELECT STATEMENT 1 48 0 (0) 00:00:01	
7 1 TABLE ACCESS BY INDEX ROWID BATCHED VENTA 1 48 0 (0) 00:00:01	
8 * 2 INDEX RANGE SCAN IDX_VENTA_CLIENTE_FECHA 1 0 (0) 00:00:01	
9	
10	
11 Predicate Information (identified by operation id):	
12	
13	
14 2 - access("V"."ID_CLIENTE"=15 AND "V"."FECHA">=TO_DATE(' 2026-01-01 00:00:00', 'syyyy-mm-dd	

13. Conclusiones

La implementación de SQL avanzado ayudó a convertir la información operativa de ILE en conocimientos aplicables para la toma de decisiones. Las subconsultas hicieron posible confrontar los resultados individuales con promedios y máximos, pero los agrupamientos fueron útiles para completar los resúmenes de ventas, compras y producción.

Las estructuras CASE y DECODE extendieron la posibilidad de clasificar clientes, proveedores e inventarios a partir de explicaciones simples, lo que mejoró la presentación de los reportes y acortó la necesidad de hacer un análisis manual de los datos.

El procedimiento almacenado congregó el registro correspondiente al detalle de una venta, la validación del inventario, así como la reducción del stock. La función simplificó la reutilización del cómputo del total de una venta al tiempo que el trigger permitió mantener los valores de la cabecera y el detalle sincronizados.

Y, por último, la redundancia controlada que se tuvo respecto a los campos subtotal y total permitiría disminuir los cálculos repetidos que surgen en sí al mismo tiempo que ayuda a mejorar los tiempos de respuesta. Su uso es seguro dado que los valores se mantienen a través del uso de procedimientos y triggers.